Memoryless Timing Is a Regime, Not a Constant

A CRR epistemic check, replacing "The Geometric Origin of Memoryless Variability" (April 2026)

Alexander Sabine · Active Inference Institute · alexander@activeinference.institute

October 2026

Abstract.

- **What this replaces.** In April 2026 I reported a "novel finding": the coefficient of variation (CV) of the exponential distribution, which is 1, could be written as (1/2π) × 2π, a product of two CRR geometric constants.
- **Why it is withdrawn.** Both constants have since been removed from CRR. The identity was in any case empty: any number times its reciprocal is 1.
- **What this paper does instead.** It asks what the *current* form of CRR says about memoryless timing. Under the current cut rule (a first passage of net forward travel of a half-turn on the carrier) and an overdamped dynamics added to it, memoryless inter-cut times appear in one regime only: when each cut is dominated by a single rare escape from a trapped state.
- **The main correction.** A CV of 1 does not identify memorylessness. Balanced drift and diffusion also gives CV ≈ 1, but with an inverse-Gaussian shape that the exponential test rejects.
- **Surplus.** In the escape regime the path is almost all surplus, and that surplus carries no information about when the cut will come.
- **How it was tested.** The predictions were frozen after development runs, anchored by a pushed commit and an OpenTimestamps stamp, then confirmed on ten fresh seeds (verdict RESULT; ceiling "PASS-0; weak anchor").
- **What it amounts to.** Everything here reduces to classical first-passage and escape theory. The contribution is a correction, and a negative control that CRR methods can use.

Keywords: Coherence–Rupture–Regeneration; first passage; inverse Gaussian; Kramers escape; memorylessness; negative control; epistemic check.

1. What the April paper claimed, and why it is withdrawn

The April paper took radioactive decay as a test case, because its waiting times are exponential and so have CV = 1. It then "derived" that 1 as

$$\mathrm{CV}_{Z2} \times \mathrm{C}^*_{SO(2)} = (1/2\pi) \times 2\pi = 1,$$

and presented this as a link between memorylessness and "missing rotational geometry".

There are three reasons it does not stand:

1. **Its inputs no longer exist.** It used separate thresholds for bistable (Z2) and rotational (SO(2)) systems, and a predicted CV = Ω/2. In the current form of CRR the threshold is Ω = 1/(half-turn), the same for every carrier. The CV = Ω/2 prediction was withdrawn after pre-registered testing.
2. **The identity was empty.** Multiplying any number by its reciprocal gives 1.
3. **CV = 1 needs no derivation.** For the exponential distribution, the mean and the standard deviation are both $1/\lambda$, so CV = 1 at every rate λ.

2. The current objects

All of these follow the current CRR core (sections R1–R4).

- **Carrier.** An intrinsic circle with Fisher metric g = 1 and circumference $\ell = 2\pi$, so the half-turn is $a = \pi$.
- **Coherence and surplus.** Coherence C is the accumulated Fisher–Rao arc. Surplus is $S = C - C^* = 2B$, where B is the total backward travel within an occasion.
- **Cut.** A first passage of net forward travel along the unwrapped path: $t_{(n+1)} = \inf\{ t > t_n : s(t) - s(t_n) = a \}$.
- **Dynamics.** Physical laws are *added* to CRR, not derived from it. Here the added law is overdamped motion with friction $\zeta = 1$, a tilt τ, a periodic potential $U(s) = A \cos 2s$ (one barrier per half-turn) and noise D: $ds = (\tau - U'(s))\, dt + \sqrt{(2D)}\, dW$.
- **Regeneration.** Each occasion restarts at the bottom of a well, so successive intervals are independent (renewal).

3. Question and method

The question. Under these rules, what produces memoryless inter-cut times, and is CV = 1 evidence that you have them?

The three regimes (`model.REGIMES`):

Regime	Tilt τ	Barrier A	Noise D	Note
Directed accumulation	1	0	0.02	
Balanced drift and diffusion	1	0	$\pi/2$	Péclet number $\tau a/D = 2$
Escape from a well	0.3	1	0.3	Kramers regime

Each regime is checked against:

- the **exact mean first-passage time**, $T = (1/D) \int_{s0}^{s0+a} dy\, e^{V(y)/D} \int_{-\infty}^{y} dz\, e^{-V(z)/D}$, with $V = U - \tau s$;
- an **exponential** with the same mean, and an **inverse Gaussian** with the same mean and variance, each by Kolmogorov–Smirnov (KS) distance;

- the **mean residual life** at the median, divided by the mean. This ratio equals 1 at every age if and only if the timing is memoryless.

Simulation. Euler–Maruyama with step dt = 0.002, 4,000 occasions per seed, and surplus sampled every 0.05 time units.

Pipeline (the Phoenix protocol; study `MEMORYLESS-CHECK-2026-10-27`):

1. **Development.** Seeds 0–2 were used for development and registered as seen.
2. **Freeze.** The predictions, scorer and code were frozen with `freeze_claim_specific.py` (FREEZE.json sha256 `056a30cc…217a`).
3. **Anchor.** The freeze commit `52c8f75` was pushed, and the freeze record was stamped with OpenTimestamps; the Bitcoin attestation is pending.
4. **Holdout.** Only after this were the holdout seeds 1000–1009 generated. Two reruns reproduced the outputs byte for byte.
5. **Ceiling.** No owner-signed tag preceded the run, so the evidential ceiling is "PASS-0; weak anchor".
6. **Instrument controls**, scored before the freeze:
    - true exponential samples meet the memoryless criteria;
    - inverse-Gaussian samples with CV = 1 do not.

4. Results (holdout seeds 1000–1009)

Table 1. Ranges across the ten holdout seeds, n = 4,000 each. The 5% KS critical value is 0.0215.

Regime	Exact mean	Simulated mean (10-seed average)	CV	KS vs exponential	KS vs inverse Gaussian	Residual life / mean at the median	Surplus / a
Directed	3.14	3.15	0.111–0.114	0.525–0.529	0.007–0.016	0.09–0.10	0.1
Balanced drift and diffusion	3.14	3.18	**0.963–1.035**	**0.117–0.125**	0.007–0.020	0.91–0.95	5–6
Escape from a well	323.51	324.56	**1.021–1.076**	**0.008–0.021**	0.111–0.125	**1.01–1.05**	268–282

All five frozen checks held, and the verdict was RESULT:

- exact-mean agreement within 5%: the largest error was 1.3%;
- the escape regime is memoryless;
- CV ≈ 1 without memorylessness in the balanced regime;
- uninformative surplus in the escape regime, with corr(S, T) ≥ 0.99995 on every seed;
- directed timing is not memoryless.

One margin was tight: the largest escape-regime KS distance, 0.0211, is just under the 0.0215 critical value. That is consistent with the escape regime being *nearly*, not exactly, exponential: its CV is slightly above 1, because an occasion sometimes slips back over a barrier before going forward.

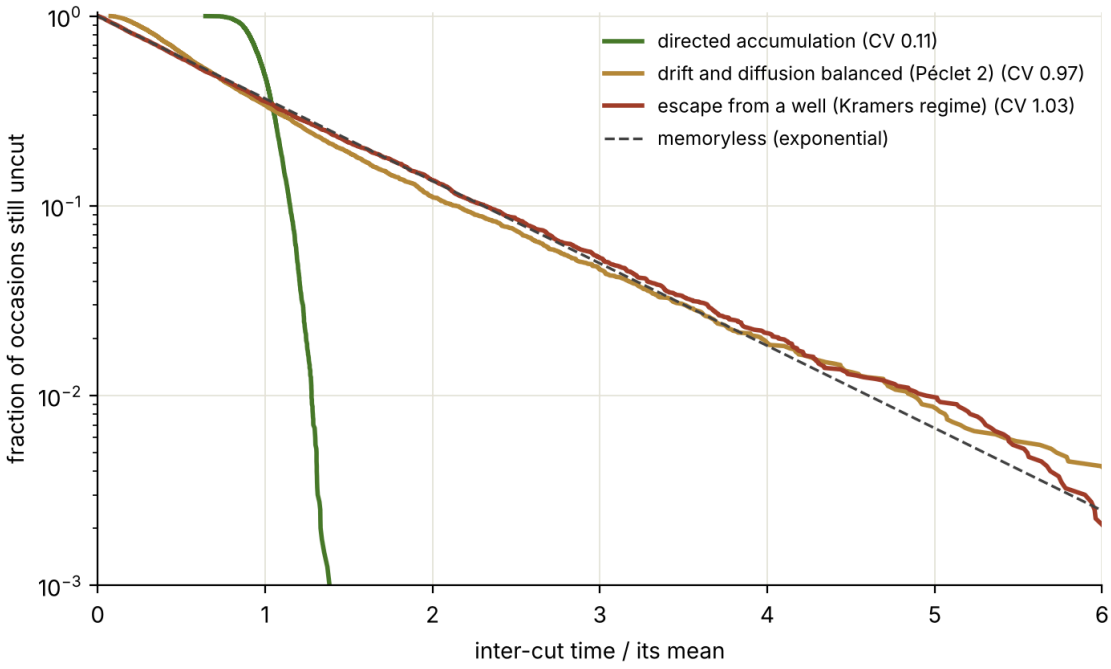


Figure 1. Fraction of occasions not yet cut against time divided by its mean, on a log scale, for holdout seed 1000. Dashed line: the exponential. The escape regime follows it. The balanced drift-and-diffusion regime has the same CV but starts flat: early cuts are rarer than memorylessness allows. Directed accumulation falls off sharply around the mean.

5. What this shows

1. **CV = 1 does not identify memorylessness.**
   - Balanced drift and diffusion gives CV ≈ 1, yet the exponential is rejected on every seed (KS ≥ 0.117) and the inverse Gaussian fits.
   - CV is one number. You cannot read a whole distribution from it, and the April paper made exactly that mistake.
2. **Under the current CRR rules, memorylessness is a regime, not a constant.**
   - The same cut rule gives directed timing (CV → 0), inverse-Gaussian timing, or escape-dominated exponential timing, depending on the added dynamics.
   - No geometric constant stands behind CV = 1.
3. **A memoryless occasion is almost all surplus, and the surplus is uninformative.**
   - In the escape regime the path travels about 270–280 half-turns in total for each half-turn of net progress, and the surplus only tracks elapsed time.
   - An occasion’s history so far says nothing about when its cut will come.
   - This is the opposite of a learning system, where surplus is written into what comes next.

4. **A negative control for CRR methods.**
    - Radioactive decay is the physical archetype of the escape regime. Gamow explained alpha decay as escape from a nuclear well, by quantum tunnelling rather than thermal noise.
    - It is therefore an ideal must-fail control. If any CRR history statistic, whether surplus, cut counts or an occasion clock, appears to predict decay times, the method is leaking information.
    - I propose this as a standing control in the Phoenix protocol.

6. What reduces to known results

Every result above is a known property of one-dimensional first passage:

- the inverse-Gaussian law for drifted Brownian motion (Schrödinger 1915);
- exponential escape over a barrier at weak noise (Kramers 1940);
- the exact passage-time integrals for tilted periodic potentials (e.g. Reimann et al. 2001).

The radioactive-decay link is structural, by way of Gamow (1928). CRR contributes only the cut rule and the surplus bookkeeping. Classification: REDUCES; novelty N0.

7. Limits

- **Surplus on a noisy path depends on resolution.** In exploratory runs of the escape regime it was 441, 272 and 121 half-turns at sampling steps of 0.02, 0.05 and 0.2. Only the pattern (surplus ≫ a, and uninformative) is robust, which is why the CRR core requires a calibrated resolution σ.
- **One potential shape and one renewal rule were tested.** Regeneration that carries history between occasions breaks independence and could move the CV in either direction.
- **The model is classical overdamped diffusion,** not nuclear physics.
- **Euler–Maruyama introduces a bias of order dt.** The largest mean error was 1.3%, in the balanced regime.
- **The operator is not independent.** The same model wrote and ran the study. The evidential ceiling is “PASS-0; weak anchor”, and any ledger entry awaits the owner’s review.

8. Reproduction

Python 3.13, numpy 2.5.3 and scipy 1.18.1 (matplotlib for the figure). From the study directory:

```
sh run.sh RUNS_DIR            # holdout seeds 1000-1009: outcomes.json and verdict.json (about 6
minutes on 4 cores)
python3 -I controls.py        # instrument controls
python3 figure.py             # Figure 1 (seed 1000)
```

All code is listed in full in the Appendix and lives in the Phoenix repository under `Phoenix_Egg/studies/MEMORYLESS-CHECK-2026-10-27/`. The receipts (freeze, anchor, exposure, reruns) are under `Phoenix_Egg/receipts/MEMORYLESS-CHECK-2026-10-27/`.

References

- Gamow, G. (1928). Zur Quantentheorie des Atomkernes. *Zeitschrift für Physik* 51, 204–212.
- Kramers, H. A. (1940). Brownian motion in a field of force and the diffusion model of chemical reactions. *Physica* 7, 284 ff.
- Reimann, P., Van den Broeck, C., Linke, H., Hänggi, P., Rubi, J. M. & Pérez-Madrid, A. (2001). Giant acceleration of free diffusion by use of tilted periodic potentials. *Physical Review Letters* 87, 010602.
- Schrödinger, E. (1915). Zur Theorie der Fall- und Steigversuche an Teilchen mit Brownscher Bewegung. *Physikalische Zeitschrift* 16, 289–295.
- Sabine, A. (2026). The Geometric Origin of Memoryless Variability (April 2026; withdrawn and replaced by this paper).

Appendix: complete code

Each file is reproduced verbatim from the frozen study directory.

`model.py`

```
        """Model for MEMORYLESS-CHECK-2026-10-27 (frozen). Inter-cut times under the current CRR
cut rule.

        Carrier: intrinsic circle, Fisher metric g = 1, circumference ell = 2*pi, half-turn a = pi
(CORE R1).
        Cut:     t_(n+1) = inf{ t > t_n : s_t - s_(t_n) = a }   (net forward travel; CORE R2).
        Law:     ds = (tau - U'(s)) dt + sqrt(2 D) dW,  U(s) = A cos(2 s)   (CORE R4, overdamped,
zeta = 1).
        Renewal: every occasion starts at the bottom of a well (s0 = pi/2 when A > 0, else 0).
        Surplus: S = 2 B, B = backward travel sampled every `res` time units (CORE R3 at a stated
resolution).
        """
        import numpy as np
        from scipy import stats, integrate

        a = np.pi
        REGIMES = {
            "directed":        dict(tau=1.0, A=0.0, D=0.02),
            "drift_diffusion": dict(tau=1.0, A=0.0, D=np.pi / 2),   # Peclet number tau*a/D = 2
            "escape":          dict(tau=0.3, A=1.0, D=0.30),        # Kramers regime: one barrier
per half-turn
        }
        DT, RES, TMAX = 2e-3, 0.05, 20000.0


        def start(p):
            return np.pi / 2 if p["A"] > 0 else 0.0


        def simulate(tau, A, D, n, seed, s0, dt=DT, res=RES, tmax=TMAX):
            rng = np.random.default_rng(seed)
            s = np.full(n, s0); done = np.zeros(n, bool); T = np.full(n, np.nan)
            last = s.copy(); back = np.zeros(n); every = max(1, int(round(res / dt))); k = 0
            while not done.all() and k * dt < tmax:
                live = ~done
                s[live] += (tau + 2 * A * np.sin(2 * s[live])) * dt + np.sqrt(2 * D * dt) *
rng.standard_normal(live.sum())
                k += 1
                if k % every == 0:
                    back += np.where(live, np.maximum(last - s, 0), 0); last = s.copy()
                hit = live & (s - s0 >= a)
                T[hit] = k * dt; done |= hit
            return T, 2 * back, float((~done).mean())


        def exact_mean(tau, A, D, s0):
            """T = (1/D) int_{s0}^{s0+a} dy e^{V(y)/D} int_{-inf}^{y} dz e^{-V(z)/D},  V = U - tau
s."""
            V = lambda x: A * np.cos(2 * x) - tau * x
            lo = s0 - 60 * D / max(tau, 1e-9) - 20
            inner = lambda y: integrate.quad(lambda z: np.exp(-(V(z) - V(y)) / D), lo, y,
limit=400)[0]
            return integrate.quad(inner, s0, s0 + a, limit=200)[0] / D


        def describe(T):
            """Distribution diagnostics used by the frozen scorer."""
            T = np.asarray(T, float); T = T[~np.isnan(T)]; m = T.mean()
            q = np.quantile(T, [0.25, 0.5, 0.75])
```

```
            lam = m ** 3 / T.var()
            return dict(n=int(T.size), mean=float(m), cv=float(T.std() / m),
                        mrl=[float((T[T > x] - x).mean() / m) for x in q],
                        ks_exp=float(stats.kstest(T, 'expon', args=(0, m)).statistic),
                        ks_ig=float(stats.kstest(T, stats.invgauss(m / lam,
scale=lam).cdf).statistic))
```

run_seeds.py

```
        """Run the three regimes on a list of seeds and write outcomes JSON (deterministic; values
rounded to 6 d.p.).
        Usage: python3 -I run_seeds.py OUT.json N SEED [SEED ...]"""
        import json, sys, pathlib
        sys.path.insert(0, str(pathlib.Path(__file__).resolve().parent))
        from multiprocessing import Pool
        import numpy as np
        from model import REGIMES, simulate, start, describe, exact_mean, a

        def one(job):
            name, seed, n = job; p = REGIMES[name]
            T, S, cens = simulate(**p, n=n, seed=seed, s0=start(p))
            d = describe(T); ok = ~np.isnan(T)
            d.update(seed=seed, censored=cens, surplus_over_a=float(np.mean(S[ok]) / a),
corr_S_T=float(np.corrcoef(S[ok], T[ok])[0, 1]))
            return name, d

        def rnd(x):
            if isinstance(x, float): return round(x, 6)
            if isinstance(x, list): return [rnd(v) for v in x]
            if isinstance(x, dict): return {k: rnd(v) for k, v in x.items()}
            return x

        if __name__ == "__main__":
            out, n, seeds = sys.argv[1], int(sys.argv[2]), [int(s) for s in sys.argv[3:]]
            jobs = [(name, s, n) for name in REGIMES for s in seeds]
            with Pool(4) as pool: res = pool.map(one, jobs)
            o = {name: {"params": REGIMES[name], "exact_mean": exact_mean(**REGIMES[name],
s0=start(REGIMES[name])),
                        "seeds": [d for nm, d in res if nm == name]} for name in REGIMES}
            pathlib.Path(out).write_text(json.dumps(rnd(o), indent=1, sort_keys=True) + "\n")
```

score.py

```
        """Frozen scorer for MEMORYLESS-CHECK-2026-10-27. Usage: python3 -I score.py outcomes.json
> verdict.json
        Rules fixed before the holdout seeds were generated (see PREREG.md). n = 4000 per seed, 10
seeds; KS 5% critical value 1.358/sqrt(n)."""
        import json, math, sys
        o = json.load(open(sys.argv[1])); crit = 1.358 / math.sqrt(4000)
        E, B, D = o["escape"]["seeds"], o["drift_diffusion"]["seeds"], o["directed"]["seeds"]
        med = lambda xs: sorted(xs)[len(xs) // 2] if len(xs) % 2 else 0.5 * (sorted(xs)[len(xs) //
2 - 1] + sorted(xs)[len(xs) // 2])
        rel = lambda reg: abs(sum(s["mean"] for s in o[reg]["seeds"]) / len(o[reg]["seeds"]) -
o[reg]["exact_mean"]) / o[reg]["exact_mean"]
        checks = {
          "V0_harness_exact_mean_within_5pct": all(rel(r) <= 0.05 for r in o) and all(s["censored"]
== 0 for r in o for s in o[r]["seeds"]) and all(s["n"] == 4000 for r in o for s in o[r]["seeds"]),
          "H1_escape_memoryless": med([s["ks_exp"] for s in E]) <= crit and sum(s["ks_exp"] <= crit
for s in E) >= 7
                                  and all(0.95 <= s["mrl"][1] <= 1.10 for s in E) and
all(s["ks_ig"] > 0.05 for s in E),
          "H2_cv1_not_memoryless": all(0.95 <= s["cv"] <= 1.05 for s in B) and all(s["ks_exp"] >
0.05 for s in B)
                                   and sum(s["ks_ig"] <= 0.04 for s in B) >= 9,
          "H3_escape_surplus_uninformative": all(s["corr_S_T"] >= 0.99 for s in E) and
```

```
all(s["surplus_over_a"] >= 50 for s in E),
          "H4_directed_not_memoryless": all(s["cv"] <= 0.2 for s in D) and all(s["ks_exp"] > 0.3
for s in D),
        }
        verdict = "VOID" if not checks["V0_harness_exact_mean_within_5pct"] else ("RESULT" if
all(checks.values()) else "FAIL")
        json.dump({"study_id": "MEMORYLESS-CHECK-2026-10-27", "ks_crit": round(crit, 6), "checks":
checks,
                   "relative_mean_error": {r: round(rel(r), 6) for r in o}, "verdict": verdict},
sys.stdout, indent=1, sort_keys=True)
        print()
```

controls.py

```
        """Instrument controls for the frozen scorer's distribution tests, scored before the
freeze.
        pos_exponential: true exponential samples must pass the memoryless criteria of H1.
        neg_inverse_gaussian_cv1: inverse-Gaussian samples with CV = 1 must fail them (the H2
trap).
        Usage: python3 -I controls.py > controls.json"""
        import json, math, sys, pathlib
        sys.path.insert(0, str(pathlib.Path(__file__).resolve().parent))
        import numpy as np
        from scipy import stats
        from model import describe
        crit = 1.358 / math.sqrt(4000); out = {}
        for name, draw in [("pos_exponential", lambda r: r.exponential(1.0, 4000)),
                           ("neg_inverse_gaussian_cv1", lambda r: stats.invgauss(1.0,
scale=1.0).rvs(4000, random_state=r))]:
            rows = [describe(draw(np.random.default_rng(7000 + i))) for i in range(10)]
            memoryless = sum(d["ks_exp"] <= crit for d in rows) >= 7 and all(0.95 <= d["mrl"][1] <=
1.10 for d in rows)
            out[name] = {"memoryless_criteria_met": bool(memoryless),
                         "ks_exp": [round(d["ks_exp"], 4) for d in rows], "cv": [round(d["cv"], 3)
for d in rows],
                         "mrl_median": [round(d["mrl"][1], 3) for d in rows]}
        out["expected"] = {"pos_exponential": True, "neg_inverse_gaussian_cv1": False}
        out["controls_behave"] = out["pos_exponential"]["memoryless_criteria_met"] and not
out["neg_inverse_gaussian_cv1"]["memoryless_criteria_met"]
        print(json.dumps(out, indent=1, sort_keys=True))
```

run.sh

```
        #!/bin/sh
        # usage: sh run.sh RUNS_DIR   (after the freeze and anchor; generates the holdout seeds
1000-1009 for the first time)
        set -e
        H="$(cd "$(dirname "$0")" && pwd)"; R="$1"; mkdir -p "$R"
        export PYTHONDONTWRITEBYTECODE=1
        python3 -I "$H/run_seeds.py" "$R/outcomes.json" 4000 1000 1001 1002 1003 1004 1005 1006
1007 1008 1009
        python3 -I "$H/score.py" "$R/outcomes.json" > "$R/verdict.json"
```

figure.py

```
        """Figure 1: survival of the inter-cut time in the three regimes, holdout seed 1000
(identical draws to the frozen run).
        Usage: python3 figure.py   (writes ../survival.png)"""
        import numpy as np
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        from model import REGIMES, simulate, start
```

```
        C = {"directed": "#4A7A2C", "drift_diffusion": "#B8893A", "escape": "#A3402B"}
        LAB = {"directed": "directed accumulation", "drift_diffusion": "drift and diffusion
balanced (Péclet 2)", "escape": "escape from a well (Kramers regime)"}
        fig, ax = plt.subplots(figsize=(7.2, 4.4), dpi=200)
        for name, p in REGIMES.items():
            T, _, _ = simulate(**p, n=4000, seed=1000, s0=start(p))
            x = np.sort(T / np.nanmean(T)); surv = 1 - np.arange(1, x.size + 1) / x.size
            cv = np.nanstd(T) / np.nanmean(T)
            ax.semilogy(x[:-1], surv[:-1], color=C[name], lw=2, label=f"{LAB[name]} (CV {cv:.2f})")
        xx = np.linspace(0, 6, 100); ax.semilogy(xx, np.exp(-xx), color="#444", lw=1.2, ls="--",
label="memoryless (exponential)")
        ax.set_xlim(0, 6); ax.set_ylim(1e-3, 1.05)
        ax.set_xlabel("inter-cut time / its mean"); ax.set_ylabel("fraction of occasions still
uncut")
        for sp in ("top", "right"): ax.spines[sp].set_visible(False)
        ax.grid(color="#e6e2d8", lw=0.6); ax.legend(frameon=False, fontsize=8.5)
        fig.tight_layout(); fig.savefig("../survival.png", facecolor="white")
```